

Marco 1 — Projeto Incremental

Bancos de Dados NoSQL: Persistência e Distribuição de Big Data

Grupo: Centauros (3 alunos)

Campus: Taguatinga

Data de Entrega: 29/09/2026

CEUB Music — Plataforma de Streaming de Música

1. Descrição do Problema

1.1 Contexto da Aplicação

O **CEUB Music** é uma plataforma fictícia de streaming de música voltada à comunidade universitária, inspirada em serviços como Spotify e Deezer. A plataforma permite que estudantes e funcionários dos campi do CEUB descubram, ouçam e organizem músicas em playlists personalizadas, enquanto acompanham o que está em alta entre seus colegas em tempo real.

O sistema precisa suportar **quatro operações centrais** que formam o núcleo da experiência do usuário:

1. **Sessão de reprodução em tempo real** — quando um usuário dá play em uma música, o sistema registra a reprodução e mantém o estado da sessão ativa (música atual, posição no tempo, fila de

reprodução). Essa informação é volátil: precisa estar disponível instantaneamente para o player do usuário, mas não precisa sobreviver por semanas caso a sessão expire.

2. **Histórico de reproduções por usuário** — toda música ouvida é registrada em um log persistente, permitindo que o usuário acesse seu histórico ("ouvidas recentemente") e que o sistema alimente algoritmos futuros de recomendação. Esse histórico cresce continuamente e precisa ser consultado por usuário e por período de tempo.
3. **Rankings e contadores em tempo real** — o sistema mantém rankings atualizados das músicas mais ouvidas globalmente, por gênero e por período (diário, semanal, mensal). Esses rankings alimentam a tela "Em Alta" da plataforma e precisam ser atualizados a cada nova reprodução, sem que a atualização bloqueie a experiência do usuário.
4. **Playlists do usuário** — cada usuário pode criar, editar e organizar múltiplas playlists. Cada playlist contém uma lista ordenada de músicas com metadados básicos (nome, artista, duração). A playlist precisa ser lida rapidamente como um bloco único (para exibir na tela do player), e alterações como reordenar ou adicionar faixas devem ser leves.

1.2 Requisitos Principais

Requisito	Descrição
Alta disponibilidade do player	O usuário nunca deve ver um erro ao dar play — a sessão e a fila de reprodução devem estar sempre acessíveis, mesmo que uma réplica esteja temporariamente fora.
Escritas de alta frequência	Cada play gera pelo menos 3 escritas simultâneas: sessão ativa, histórico e incremento no ranking. Em horários de pico, isso multiplica rapidamente.
Leitura analítica por período	O painel "Em Alta" e o histórico do usuário são consultas previsíveis: "Top 50 da semana", "últimas 20 músicas ouvidas pelo usuário X".
Escalabilidade horizontal	O sistema deve suportar crescimento de usuários e de catálogo sem redesenho — adicionar nós, não trocar o servidor.
Baixa latência no player	Operações que afetam a experiência em tempo real (play, pause, próxima faixa) precisam de respostas em milissegundos, não em centenas de milissegundos.

1.3 Estimativa de Volume de Dados

A plataforma é dimensionada para o primeiro ano de operação na comunidade CEUB:

Métrica	Estimativa
Usuários ativos	8.000 (alunos + funcionários dos campi)
Catálogo de músicas	500.000 faixas
Reproduções por dia (média)	~40.000 (média de 5 músicas/dia por usuário ativo)
Reproduções por dia (pico)	~80.000 (intervalos de aula, almoço)
Pico de escritas por segundo	~200 escritas/s em janelas de 10 minutos nos horários de pico
Playlists por usuário (média)	4 playlists, ~25 músicas cada
Crescimento do histórico	~14,6 milhões de registros/ano
Tamanho médio de um registro de reprodução	~200 bytes → ~2,9 GB/ano de histórico bruto

Os horários de pico são concentrados: **intervalos de aula (9h50–10h10, 11h30–11h50)** e **horário de almoço (12h–13h30)** acumulam mais de 60% do tráfego diário em janelas curtas, gerando picos de escritas muito superiores à média.

2. Modelagem Inicial dos Dados

O grupo optou por combinar **duas famílias NoSQL**, cada uma aplicada ao tipo de dado e padrão de acesso para o qual foi projetada:

- **Chave-Valor (Redis)** — para dados voláteis, de acesso ultrarrápido e por chave única: sessão de reprodução, fila do player, playlists e rankings em tempo real.
- **Wide-Column (Cassandra)** — para dados duráveis, de grande volume e consultados por partição + intervalo temporal: histórico de reproduções e logs analíticos.

2.1 Dados em Chave-Valor — Redis

Sessão de reprodução ativa (Hash com TTL)

```
sessao:usuario:5021
  musica_atual    → "track:88201"
  posicao_seg      → 147
  status          → "playing"
  inicio          → "2026-09-28T14:32:00Z"

TTL: 7200s (2 horas – expira se o usuário ficar inativo)
```

Justificativa da estrutura: o Hash do Redis agrupa todos os campos da sessão sob uma única chave, permitindo ler ou atualizar campos individuais (`HGET` , `HSET`) sem transferir o objeto inteiro. O TTL garante limpeza automática de sessões abandonadas — implementando o conceito de **soft state** do modelo BASE.

Fila de reprodução do usuário (List)

```
fila:usuario:5021
[0] "track:44102"
[1] "track:77503"
[2] "track:91204"
[3] "track:33067"
```

Justificativa: a List do Redis suporta `LPUSH` , `RPUSH` , `LPOP` e acesso por índice, modelando naturalmente uma fila de reprodução onde faixas são adicionadas ao final e consumidas pela frente (FIFO), ou reordenadas pelo usuário.

Playlist do usuário (Hash)

```
playlist:usuario:5021:meu_rock
  nome          → "Rock para estudar"
  criada_em     → "2026-08-15T10:00:00Z"
  total_faixas  → 3

playlist:faixas:usuario:5021:meu_rock
0 → {"track_id": "track:88201", "nome": "Bohemian Rhapsody", "ar"}
1 → {"track_id": "track:44102", "nome": "Stairway to Heaven", "a"}
2 → {"track_id": "track:91204", "nome": "Hotel California", "ar"}
```

Justificativa: separar metadados da playlist (Hash) das faixas (Hash ordenado por índice numérico) permite listar playlists do usuário sem carregar todas as faixas. A leitura completa da playlist para o player usa um único `HGETALL`.

Rankings em tempo real (Sorted Set)

```
ranking:global:diario:2026-09-28
  ZADD track:88201 1427 → (1.427 reproduções no dia)
  ZADD track:44102 1183 → (1.183 reproduções no dia)
  ZADD track:77503 892 → (892 reproduções no dia)

ranking:genero:rock:semanal:2026-W39
  ZADD track:88201 8340
  ZADD track:44102 6721

ranking:global:mensal:2026-09
  ZADD track:88201 41200
```

Justificativa: o **Sorted Set** do Redis é a estrutura ideal para rankings — cada `ZINCRBY` incrementa o score de uma faixa em $O(\log N)$ a cada reprodução, e o `ZREVRANGE` retorna o Top N em tempo real sem varredura. As chaves são particionadas por período (diário, semanal, mensal), permitindo expirar rankings antigos automaticamente com TTL e evitando sets de tamanho ilimitado.

Contadores atômicos simples (String)

```
plays:total:track:88201 → 234.567 (INCR a cada reprodução)
plays:total:artista:queen → 1.892.043 (INCR a cada reprodução)
```

Justificativa: o `INCR` do Redis é atômico e executa em $O(1)$, suportando centenas de incrementos por segundo sem contenção — ideal para contadores de reprodução que atualizam a cada play.

2.2 Dados em Wide-Column — Cassandra

Histórico de reproduções por usuário

```
CREATE TABLE historico_por_usuario (  
    usuario_id INT,  
    reproduzido_em TIMESTAMP,  
    track_id TEXT,  
    nome_musica TEXT,  
    artista TEXT,  
    duracao_seg INT,  
    dispositivo TEXT,  
    PRIMARY KEY (usuario_id, reproduzido_em)  
    ) WITH CLUSTERING ORDER BY (reproduzido_em DESC);
```

Modelagem query-first: a consulta mais frequente é *"últimas N músicas ouvidas pelo usuário X"*:

```
SELECT * FROM historico_por_usuario  
WHERE usuario_id = 5021  
ORDER BY reproduzido_em DESC  
LIMIT 20;
```

- **Partition key:** `usuario_id` — garante que todo o histórico de um usuário reside no mesmo nó, eliminando consultas cross-partition.
- **Clustering column:** `reproduzido_em DESC` — os dados já são armazenados em ordem cronológica inversa no disco, tornando a consulta "últimas N" uma leitura sequencial eficiente sem ordenação em runtime.

Reproduções por artista (para analytics)

```
CREATE TABLE reproducoes_por_artista (  
    artista_id INT,  
    reproduzido_em TIMESTAMP,  
    track_id TEXT,  
    nome_musica TEXT,  
    usuario_id INT,  
    PRIMARY KEY (artista_id, reproduzido_em)  
    ) WITH CLUSTERING ORDER BY (reproduzido_em DESC);
```

Modelagem query-first: essa tabela atende a consulta *"reproduções de um artista em um período"*, usada pelo painel de analytics para mostrar

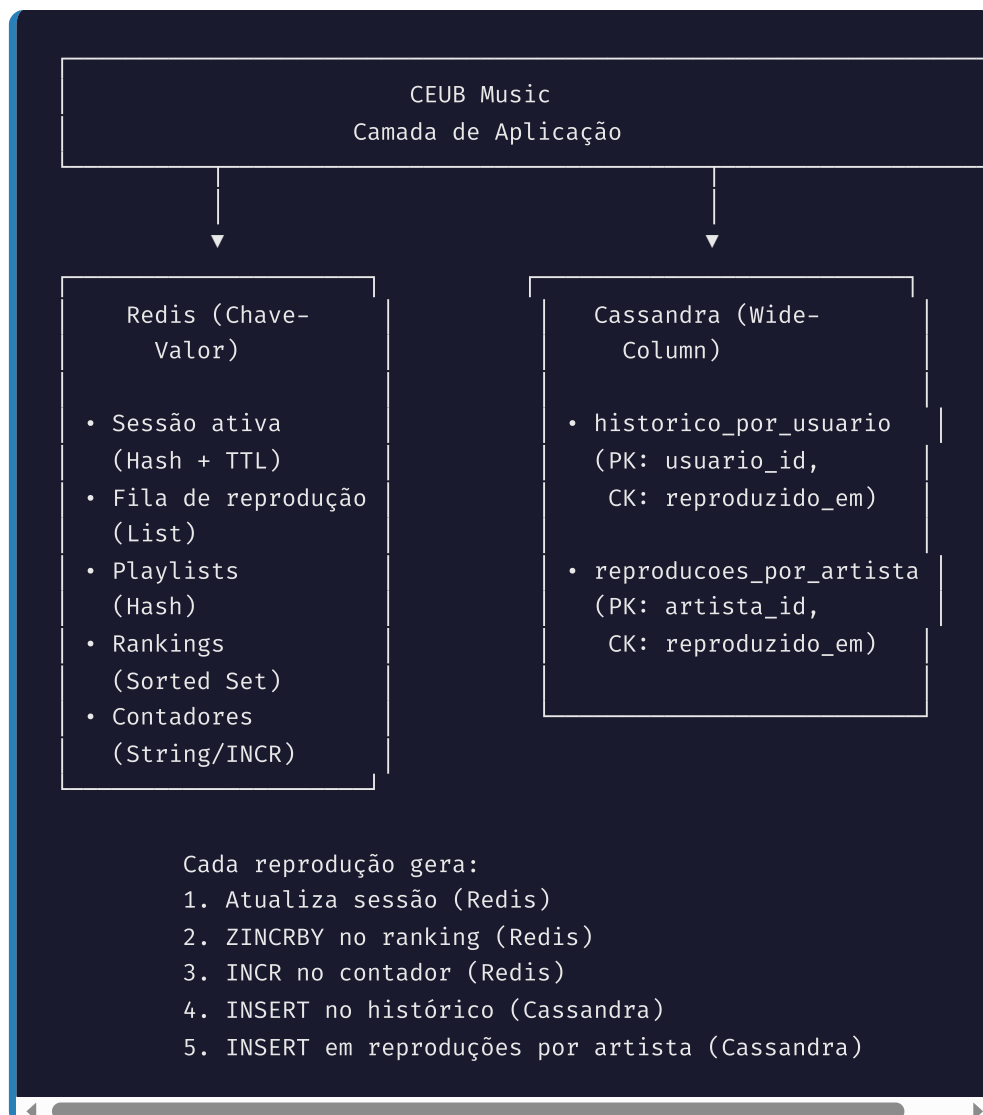
tendências:

```
SELECT * FROM reproducoes_por_artista
WHERE artista_id = 42
AND reproduzido_em ≥ '2026-09-01'
AND reproduzido_em ≤ '2026-09-30';
```

- **Partition key:** `artista_id` — agrupa todas as reproduções de um artista no mesmo nó.
- **Clustering column:** `reproduzido_em DESC` — permite range queries eficientes por período.

Nota sobre desnormalização: no modelo Wide-Column, a mesma reprodução pode existir em mais de uma tabela (histórico do usuário e reproduções por artista). Isso é intencional e esperado: a modelagem é orientada às consultas, não às entidades — diferentemente do modelo relacional, onde normalizaríamos para evitar redundância. O custo de armazenamento extra é pequeno frente ao ganho de performance na leitura.

2.3 Diagrama da Arquitetura de Dados



3. Justificativa Tecnológica

3.1 Limites do Modelo Relacional (Unidade 1)

Um banco relacional tradicional atenderia o CEUB Music em cenários de baixo tráfego, mas apresentaria três gargalos significativos conforme o sistema escala:

1. **Custo de joins sob carga concorrente** — montar a tela "Em Alta" em um banco relacional exigiria um `JOIN` entre tabelas de reproduções, músicas e artistas, seguido de `GROUP BY` e `ORDER BY COUNT(*)`. Sob pico de 200 escritas/segundo, essa consulta

análítica concorreria com as inserções de reprodução no mesmo servidor, degradando a latência de ambas as operações.

2. **Contadores e rankings em tabelas transacionais** — incrementar um contador de reproduções em uma tabela relacional com `UPDATE plays SET count = count + 1 WHERE track_id = ?` causa contenção de locks por linha quando centenas de usuários ouvem a mesma música simultaneamente. O `INCR` do Redis, por ser atômico e single-threaded, elimina completamente esse problema.
3. **Escalabilidade vertical como único caminho** — bancos relacionais escalam primordialmente de forma vertical (mais CPU, mais RAM no mesmo servidor). O volume projetado de ~14,6 milhões de registros/ano de histórico, com escritas concentradas em janelas de pico, favorece a escalabilidade horizontal nativa de bancos distribuídos como o Cassandra, onde adicionar nós ao cluster aumenta linearmente a capacidade de escrita.

3.2 Teorema CAP (Unidade 1)

O sistema CEUB Music faz **escolhas de consistência diferentes conforme o tipo de dado**, alinhando-se ao Teorema CAP de forma intencional:

Componente	Classificação CAP	Justificativa
Sessão de reprodução (Redis)	AP (Disponibilidade)	Se o usuário aperta play e o sistema está indisponível, a experiência é quebrada. É preferível servir um estado de sessão momentaneamente desatualizado (ex.: posição da música atrasada em 1 segundo) do que retornar um erro.
Rankings em tempo real (Redis)	AP (Disponibilidade)	O ranking "Em Alta" pode estar 30 segundos desatualizado sem que o usuário perceba — não há impacto funcional. A disponibilidade do ranking é mais importante que sua precisão absoluta.
Histórico de reproduções (Cassandra)	AP (Disponibilidade)	O Cassandra é projetado como sistema AP por padrão: em caso de partição de rede, os nós continuam aceitando escritas e se reconciliam depois. Para o histórico, isso é adequado — uma reprodução registrada com atraso de segundos entre réplicas não causa inconsistência funcional.

Por que não CP (Consistência forte)? Nenhum dos dados do CEUB Music exige consistência forte imediata — não há transações financeiras, não há inventário com risco de overselling. O pior cenário de inconsistência eventual é um ranking momentaneamente desatualizado ou uma reprodução que aparece no histórico com alguns segundos de atraso — ambos imperceptíveis para o usuário.

3.3 ACID vs. BASE (Unidade 1)

O CEUB Music adota o modelo **BASE (Basically Available, Soft State, Eventually Consistent)** para todos os seus componentes:

- **Basically Available** — o sistema continua operacional mesmo que partes estejam degradadas. Se um nó do Redis cai, o Sentinel

promove uma réplica automaticamente; se um nó do Cassandra fica indisponível, os demais nós do cluster assumem as partições afetadas.

- **Soft State** — o estado do sistema muda sem intervenção explícita do usuário:
- A sessão de reprodução no Redis expira automaticamente após 2 horas de inatividade (TTL).
- Rankings diários expiram ao final do dia, dando lugar ao ranking do dia seguinte.
- Contadores em réplicas diferentes podem ter valores momentaneamente distintos.
- **Eventually Consistent** — após uma escrita no Cassandra, nem todas as réplicas refletem o novo dado imediatamente, mas convergem em segundos. Para o histórico de reproduções, isso é aceitável: o usuário não percebe se a última música ouvida aparece no histórico com 2 segundos de atraso.

Por que não ACID? Garantias ACID (transações distribuídas, isolamento completo) adicionariam latência e complexidade incompatíveis com o requisito de baixa latência do player. Um `BEGIN TRANSACTION` para cada reprodução — abrangendo sessão, ranking e histórico — seria desnecessariamente custoso para dados que não requerem isolamento transacional.

3.4 Sharding e Replicação (Unidade 1)

Estratégia de Sharding

Banco	Estratégia	Chave de Partição	Justificativa
Cassandra	Hash-based (Murmur3 partitioner)	<code>usuario_id</code> (histórico) / <code>artista_id</code> (analytics)	O particionador hash do Cassandra distribui automaticamente os dados entre os nós do cluster usando hash da partition key. Isso garante distribuição uniforme — nenhum nó acumula desproporcionalmente os dados de usuários populares.
Redis	Consistent Hashing (Redis Cluster)	Prefixo da chave (ex.: <code>sessao:usuario:5021</code>)	O Redis Cluster usa 16.384 hash slots distribuídos entre os nós. As chaves são atribuídas a slots via CRC16, e o Consistent Hashing garante que adicionar ou remover nós afeta no máximo 1/N das chaves — minimizando a redistribuição.

Estratégia de Replicação

Banco	Estratégia	Fator	Justificativa
Cassandra	Master-Master (peer-to-peer, sem nó primário)	Fator de replicação = 3	O Cassandra replica cada partição em 3 nós — qualquer um aceita escritas, sem ponto único de falha. Isso é essencial para o CEUB Music: durante o pico de almoço, a escrita de reproduções não pode depender de um único nó master.
Redis	Master-Slave (Redis Sentinel)	1 master + 2 réplicas por shard	Para sessões e rankings, a escrita é direcionada ao master e replicada assincronamente para as réplicas. As réplicas atendem leituras (ex.: consultas ao ranking), distribuindo a carga. Em caso de falha do master, o Sentinel promove uma réplica automaticamente.

Comparação com o cenário relacional: em um banco relacional, implementar sharding requer soluções externas (middleware, lógica na aplicação) e replicação Master-Master causa conflitos de escrita difíceis de resolver. No Cassandra, sharding e replicação são nativos e transparentes — a aplicação escreve normalmente e o cluster distribui e replica automaticamente.

3.5 Chave-Valor — Redis (Unidade 2)

O grupo utiliza **cinco estruturas de dados do Redis**, cada uma escolhida pelo ajuste natural ao padrão de acesso do dado:

Estrutura	Uso no CEUB Music	Por que esta estrutura
Hash	Sessão de reprodução, playlists	Agrupa campos relacionados sob uma chave; permite ler/atualizar campos individuais sem serializar o objeto inteiro.
List	Fila de reprodução	Suporta FIFO (LPUSH/RPOP) e acesso posicional — modela naturalmente uma fila ordenada de músicas.
Sorted Set	Rankings (diário, semanal, mensal, por gênero)	<code>ZINCRBY</code> incrementa o score atomicamente; <code>ZREVRANGE</code> retorna o Top N sem varredura — operações ideais para ranking.
String	Contadores de reprodução (por faixa, por artista)	<code>INCR</code> atômico em $O(1)$ — sem locks, sem contenção, mesmo sob centenas de incrementos simultâneos.
TTL	Expiração de sessões (2h) e rankings diários (24h)	Implementa soft state automaticamente, sem necessidade de jobs de limpeza — alinhado ao modelo BASE.

Consistent Hashing (Dynamo / Redis Cluster)

O **Consistent Hashing**, estudado no contexto do Dynamo (Amazon), é utilizado pelo Redis Cluster para distribuir chaves entre nós. O princípio é o mesmo: chaves e nós são mapeados em um anel hash; cada chave é atribuída ao primeiro nó encontrado no sentido horário do anel.

No CEUB Music, isso significa que ao escalar de 3 para 5 nós Redis durante o pico de almoço, apenas $\sim 2/5$ das chaves precisam ser redistribuídas — em vez de reorganizar todas, como aconteceria com hashing simples (`key % N`). Isso permite **elasticidade horizontal com mínimo de interrupção**.

3.6 Wide-Column — Cassandra (Unidade 3)

Modelagem Query-First

Diferentemente do modelo relacional, onde primeiro se definem as entidades e depois se consultam com joins, no Cassandra a modelagem parte das **consultas que a aplicação precisa fazer**:

Consulta da aplicação	Tabela modelada	Partition Key	Clustering Column
"Últimas 20 músicas ouvidas pelo usuário X"	historico_por_usuario	usuario_id	reproduzido_em DESC
"Reproduções do artista Y no mês de setembro"	reproducoes_por_artista	artista_id	reproduzido_em DESC

Cada tabela é desenhada para responder **uma consulta específica** de forma eficiente — leitura sequencial dentro de uma única partição, sem scans cross-partition.

Column Families

No modelo Wide-Column do Cassandra:

- **Column Family** = tabela CQL. Cada column family agrupa colunas logicamente relacionadas.
- **historico_por_usuario** é uma column family cujas colunas (track_id, nome_musica, artista, duracao_seg, dispositivo) representam os atributos de cada reprodução.
- **reproducoes_por_artista** é outra column family com colunas diferentes (track_id, nome_musica, usuario_id), otimizada para a perspectiva do artista.

Partition Key e Clustering Columns

- **Partition key** (**usuario_id** ou **artista_id**) — determina **em qual nó** do cluster os dados residem. O Cassandra aplica o Murmur3 hash sobre a partition key para atribuir os dados a um nó. Escolher a

partition key correta é a decisão mais impactante: ela deve garantir que os dados acessados juntos estejam na mesma partição.

- **Clustering column** (`reproduzido_em DESC`) — determina a **ordem física** dos dados dentro da partição no disco (SSTable). Ao definir `CLUSTERING ORDER BY (reproduzido_em DESC)`, o Cassandra armazena as reproduções mais recentes primeiro — a consulta "últimas 20" lê os primeiros 20 registros sequencialmente, sem ordenação em runtime.
-

4. Resumo: Por que essa combinação tecnológica

Necessidade do CEUB Music	Tecnologia	Motivo
Sessão do player com latência mínima	Redis (Hash + TTL)	Acesso por chave em memória, $O(1)$, soft state automático
Rankings atualizados a cada play	Redis (Sorted Set)	ZINCRBY atômico, ZREVRANGE para Top N em tempo real
Contadores de reprodução sem locks	Redis (String/INCR)	Operação atômica $O(1)$, sem contenção sob concorrência
Histórico durável de milhões de reproduções	Cassandra	Escrita distribuída, compactação eficiente, leitura sequencial por partição
Analytics por artista e período	Cassandra	Modelagem query-first com range queries sobre clustering column
Escalabilidade horizontal no pico	Ambos	Redis Cluster (Consistent Hashing) + Cassandra (hash partitioner) escalam adicionando nós
Alta disponibilidade sem ponto único de falha	Ambos	Redis Sentinel (failover automático) + Cassandra peer-to-peer (sem master)

5. Referências

- SADALAGE, P. J.; FOWLER, M. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley, 2013.
- DeCANDIA, G. et al. *Dynamo: Amazon's Highly Available Key-Value Store*. SOSP, 2007.

- CHANG, F. et al. *Bigtable: A Distributed Storage System for Structured Data*. OSDI, 2006.
- LAKSHMAN, A.; MALIK, P. *Cassandra: A Decentralized Structured Storage System*. LADIS, 2010.
- Redis Documentation. Disponível em: <https://redis.io/docs/>
- Apache Cassandra Documentation. Disponível em: <https://cassandra.apache.org/doc/latest/>